

APPLY THIRD NORMAL FORM (3NF) AND BCNF

1. Third Normal Form (3NF)

A database is in **Third Normal Form (3NF)** when:

1. It is already in **Second Normal Form (2NF)**.
2. There are no **transitive dependencies**.
3. Non-key attributes depend only on the primary key.

In the proposed E-Commerce database, major entities such as Customer, Seller, Category, Product, Orders, Payment, and Shipping are already separated into individual tables.

Example of Removing Transitive Dependency

Suppose the Product table contained:

Product_ID → Category_ID → Category_Name

Here, Category_Name is indirectly dependent on Product_ID through Category_ID. To avoid this transitive dependency, Category information is stored separately.

CATEGORY(

Category_ID PK,

Category_Name

)

PRODUCT(

Product_ID PK,

Product_Name,

Category_ID FK,

Seller_ID FK,

Price,

Stock_Quantity,

Product_Status

)

Similarly, seller details are stored separately and connected to Product using Seller_ID. This reduces unnecessary repetition of category and seller information.

2. Boyce-Codd Normal Form (BCNF)

A relation is in **BCNF** if, for every functional dependency:

$X \rightarrow Y$

X must be a candidate key or super key.

In the final database design, the main functional dependencies are based on primary keys.

Customer_ID $\rightarrow$ Customer_Name, Email, Phone, Address, Registration_Date

Seller_ID $\rightarrow$ Seller_Name, Email, Phone, Address

Category_ID $\rightarrow$ Category_Name

Product_ID $\rightarrow$ Product_Name, Category_ID, Seller_ID,
Price, Stock_Quantity, Product_Status

Order_ID $\rightarrow$ Customer_ID, Order_Date, Order_Status, Total_Amount

Order_Detail_ID $\rightarrow$ Order_ID, Product_ID, Quantity,
Unit_Price, Subtotal

Payment_ID $\rightarrow$ Order_ID, Payment_Method, Amount,
Payment_Date, Payment_Status

Shipping_ID $\rightarrow$ Order_ID, Shipping_Address,
Tracking_Number, Delivery_Status

Since the determinants in these dependencies are the primary keys, the relations satisfy the basic BCNF requirement.

Final Normalized Database Design

CUSTOMER

Customer_ID (PK)

Customer_Name

Email

Phone

Address

Registration_Date

SELLER

Seller_ID (PK)

Seller_Name

Email

Phone

Address

CATEGORY

Category_ID (PK)

Category_Name

PRODUCT

Product_ID (PK)

Product_Name

Category_ID (FK)

Seller_ID (FK)

Price

Stock_Quantity

Product_Status

ORDERS

Order_ID (PK)

Customer_ID (FK)

Order_Date

Order_Status

Total_Amount

ORDER_DETAILS

Order_Detail_ID (PK)

Order_ID (FK)

Product_ID (FK)

Quantity

Unit_Price

Subtotal

PAYMENT

Payment_ID (PK)

Order_ID (FK)

Payment_Method

Amount

Payment_Date

Payment_Status

SHIPPING

Shipping_ID (PK)

Order_ID (FK)

Shipping_Address

Tracking_Number

Delivery_Status

The entities, keys, and relationships in this final schema are based on the original E-Commerce Order Management System design.

Final Normalization Flow

UNNORMALIZED DATABASE

|



1NF

Atomic values

No repeating groups

|



2NF

No partial dependencies

|



3NF

No transitive dependencies

|



BCNF

Every determinant is a candidate key

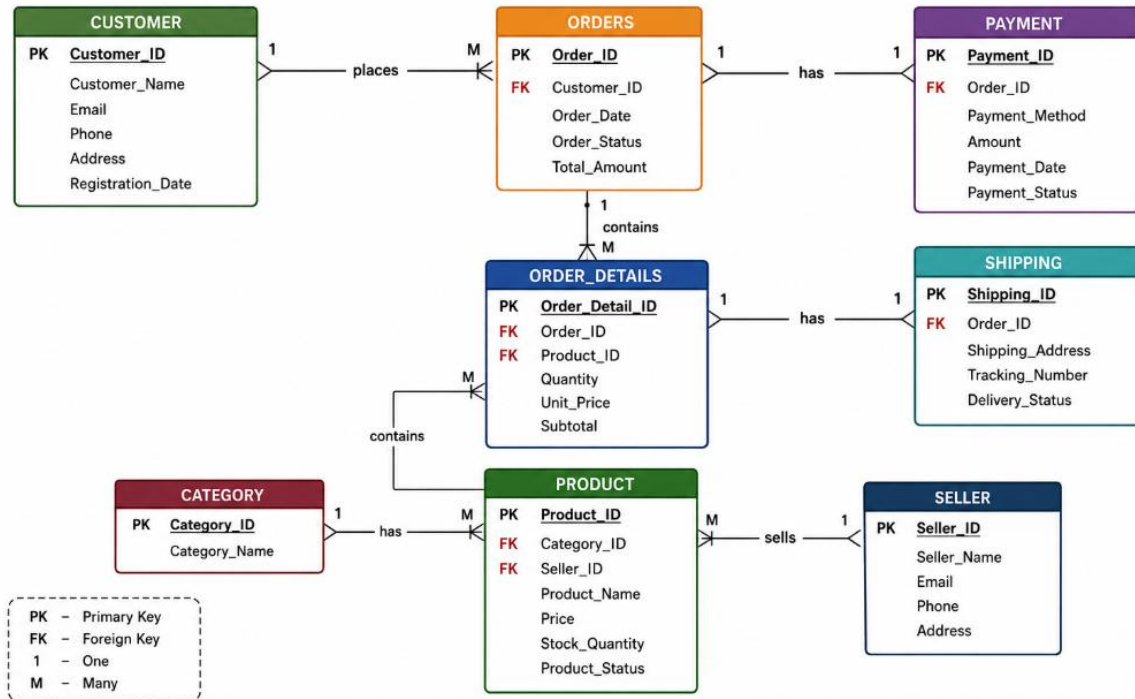
|



FINAL NORMALIZED DATABASE

STRUCTURE DIAGRAM – FINAL NORMALIZED DATABASE DESIGN (3NF / BCNF)

E-COMMERCE ORDER MANAGEMENT SYSTEM



Conclusion

The final E-Commerce Order Management database design is organized into **eight normalized relations**: Customer, Seller, Category, Product, Orders, Order_Details, Payment, and Shipping. The design separates independent entities, uses primary keys for unique identification, and uses foreign keys to maintain relationships. This minimizes data redundancy and helps prevent insertion, update, and deletion anomalies.